

Chapter 26 - The 6502 Processor

The 6502 is the small 8-bit processor that powered the home computers of the late 1970s and early 1980s: the Commodore 64, the Apple II, the BBC Micro, the Atari 8-bit family, and many others. Intuition Engine runs it natively, with cycle-accurate timing and the full documented and undocumented instruction set.

This chapter is the architectural reference for writing 6502 code on Intuition Engine: registers, addressing modes, instruction groups, vectors, and the bank-and-MMIO mapping that gives the small CPU a path to the 32-bit hardware.

26.1 Architecture

The 6502 has three 8-bit data registers, a 16-bit program counter, and an 8-bit stack pointer:

Register	Width	Purpose
A	8	Accumulator. The destination for arithmetic and logic.
X	8	Index register. Used for indexed addressing and counters.
Y	8	Index register. Same uses as X but with different addressing modes.
PC	16	Program counter.
SP	8	Stack pointer. The actual stack lives at $0x0100 + SP$, growing downward.
P	8	Status register. Flags N, V, -, B, D, I, Z, C (high bit first).

The seven P flags:

Bit	Name	Meaning
7	N	Negative - copy of bit 7 of last result
6	V	Overflow - set on signed overflow from arithmetic
5	-	Always reads 1
4	B	Break - set on BRK, used to distinguish BRK from IRQ
3	D	Decimal - when set, ADC/SBC operate in BCD
2	I	Interrupt disable - when set, the CPU ignores IRQ
1	Z	Zero - set when last result was zero
0	C	Carry - set on unsigned overflow from arithmetic and from shifts

26.2 Memory model

The 6502 sees a 64-kilobyte address space, $0x0000-0xFFFF$. Two pages have a special role:

- **Zero page** ($0x0000-0x00FF$) - addressed with a single byte; cheaper and faster to access than ordinary memory. Most variables in classic 6502 programs live here.
- **Stack** ($0x0100-0x01FF$) - the SP register names a byte in this page. Push decrements SP, pop increments SP.

The top six bytes of the address space hold the interrupt vectors:

Address	Vector	Triggered by
0xFFFFA/0xFFFFB	NMI	Non-maskable interrupt
0xFFFFC/0xFFFFD	RESET	Power-on or reset
0xFFFFE/0xFFFFF	IRQ/BRK	Maskable interrupt or BRK instr

Each vector is a 16-bit little-endian pointer to the handler.

26.3 The Intuition Engine memory map for the 6502

On a real 1980 machine the 64-kilobyte map was hard-coded by the manufacturer. Intuition Engine gives the 6502 the same 64-K view, but provides a small set of fixed apertures that map back into the larger 32-bit address space:

Range	Purpose
0x0000–0x00FF	Zero page (CPU RAM)
0x0100–0x01FF	Stack page (CPU RAM)
0x0200–0x1FFF	Free RAM
0x2000–0x3FFF	Bank 1 window (8 KiB, sprite/general data)
0x4000–0x5FFF	Bank 2 window (8 KiB, font/general data)
0x6000–0x7FFF	Bank 3 window (8 KiB, general data)
0x8000–0xBFFF	VRAM bank window (16 KiB)
0xC000–0xCFFF	Free RAM
0xD700–0xD70A	VGA registers
0xD800–0xD817	ULA registers (with paged VRAM port)
0xF000–0xFFFF9	MMIO window (maps to 0xF0000–0xF0FF9)
0xFFFFA–0xFFFFF	Vector table (not translated)

The CPU adapter additionally intercepts the bank-control bytes inside the MMIO window:

Address	Register
0xF700/0xF701	BANK1_REG_LO / BANK1_REG_HI
0xF702/0xF703	BANK2_REG_LO / BANK2_REG_HI
0xF704/0xF705	BANK3_REG_LO / BANK3_REG_HI
0xF7F0	VRAM_BANK_REG

Writes to these bytes change the backing region of the matching bank window, instead of falling through to the bus. All other addresses inside 0xF000–0xFFFF9 translate transparently; for example, 0xFC30 reaches 0xF0C30 which is SN_PORT_WRITE, and 0xF0C0 reaches 0xF00C0 which is VIDEO_PAL_BANK_4.

The 6502 program never sees the full 32-bit address; it sees a 16-K (VRAM) or 8-K (banks) piece of it at a time. There is no 16-bit alias that reaches TERM_OUT (0xF0700) directly, because the \$F700 page is captured by the bank-register intercept. Bare 6502 code that needs to display text uses the ULA or VGA chips rather than the system terminal.

26.4 Addressing modes

The 6502 has 13 addressing modes. The mnemonic conventions used throughout this chapter are:

Mode	Notation	Meaning
Implied	CLC	No operand
Accumulator	ASL A	The accumulator is the operand
Immediate	LDA #\$42	The literal byte
Zero page	LDA \$80	Byte at zero-page address
Zero page,X	LDA \$80,X	Byte at $(\$80 + X) \bmod 256$
Zero page,Y	LDX \$80,Y	Byte at $(\$80 + Y) \bmod 256$
Absolute	LDA \$1234	Byte at 0x1234
Absolute,X	LDA \$1234,X	Byte at $\$1234 + X$
Absolute,Y	LDA \$1234,Y	Byte at $\$1234 + Y$
Indirect	JMP (\$1234)	Jump to address held at \$1234/\$1235 (JMP only)
Indexed indirect	LDA (\$80,X)	Byte at the address held at $(\$80 + X) / (\$80 + X + 1)$
Indirect indexed	LDA (\$80),Y	Byte at $(\text{address held at } \$80/\$81) + Y$
Relative	BNE label	Signed 8-bit branch offset

The two indirect modes accept only zero-page operands. The indirect-jump bug of the original 6502 is preserved: a JMP (\$XXFF) fetches its high byte from \$XX00, not $(XX+1)00$.

26.5 Instruction set summary

The 6502 has 56 mnemonics. They divide into the following groups. Appendix G has the full opcode table; this section lists each mnemonic and what it does.

26.5.1 Load and store

Mnemonic	Operation
LDA	Load accumulator (sets N, Z)
LDX	Load X (sets N, Z)
LDY	Load Y (sets N, Z)
STA	Store accumulator
STX	Store X
STY	Store Y

26.5.2 Register transfers and stack

Mnemonic	Operation
TAX/TAY/TSX/TXA/TXS/TYA	Copy between registers

Mnemonic	Operation
PHA/PHP	Push A / push P
PLA/PLP	Pop into A / pop into P

26.5.3 Arithmetic and logic

Mnemonic	Operation
ADC	Add with carry ($A = A + M + C$)
SBC	Subtract with carry ($A = A - M - (1 - C)$)
AND/ORR/EOR	Bitwise AND / OR / XOR
INC/DEC	Increment / decrement memory
INX/INY/DEX/DEY	Increment / decrement index registers
ASL/LSR	Arithmetic shift left / logical shift right
ROL/ROR	Rotate left / right through carry
CMP/CPX/CPY	Compare against A/X/Y (sets N,Z,C)
BIT	Test memory bits against A (sets N,V,Z)

In decimal mode (D flag set), ADC and SBC operate on packed BCD digits.

26.5.4 Branches

Each branch is a signed 8-bit PC-relative offset, range -128 to +127:

Mnemonic	Branches if
BCC	$C = 0$
BCS	$C = 1$
BEQ	$Z = 1$
BNE	$Z = 0$
BMI	$N = 1$
BPL	$N = 0$
BVC	$V = 0$
BVS	$V = 1$

26.5.5 Jumps and subroutines

Mnemonic	Operation
JMP	Unconditional jump (absolute or indirect)
JSR	Jump to subroutine (push return - 1)
RTS	Return from subroutine
BRK	Software interrupt (push PC, push P, set B, jump via 0xFFFE/0xFFFF)

Mnemonic	Operation
RTI	Return from interrupt (pop P, pop PC)

26.5.6 Flag operations

Mnemonic	Operation
CLC/SEC	Clear / set carry
CLI/SEI	Clear / set interrupt disable
CLV	Clear overflow
CLD/SED	Clear / set decimal mode
NOP	No operation

26.6 Undocumented opcodes

The original NMOS 6502 has 105 unintended opcodes that the silicon nevertheless decodes into observable behavior. Intuition Engine's 6502 implements every one of them so that classic software using them continues to work. Their mnemonics in common use include SLO, RLA, SRE, RRA, SAX, LAX, DCP, ISC, and a few others. The full list is in Appendix G. These are best avoided in fresh code; they exist for compatibility with software that already relies on them.

26.7 Interrupts on Intuition Engine

When the IRQ line is asserted and the I flag is clear:

1. The current PC is pushed (high byte first, then low).
2. P is pushed with the B flag clear.
3. I is set.
4. PC is loaded from 0xFFFF/0xFFFF.

NMI is identical but uses 0xFFFA/0xFFFB and ignores the I flag. RESET is identical but uses 0xFFFC/0xFFFD, sets I, and does not push anything (it uses the stack page as scratch).

BRK works like an IRQ but pushes P with the B flag set, so the handler can distinguish a software trap from a hardware interrupt.

26.8 A small example

A complete 6502 program that paints a red pixel on the ULA at (8, 8) and then spins:

```

ULA_CTRL      = $D801
ULA_ADDR_LO   = $D803
ULA_ADDR_HI   = $D805
ULA_DATA      = $D807

        .org   $1000

start:  lda    #$01          ; enable the ULA
        sta    ULA_CTRL

        lda    #$01          ; row 8 bitmap address low: see ch 8 formula
        sta    ULA_ADDR_LO
        lda    #$00
        sta    ULA_ADDR_HI
        lda    #$80          ; bit 7 -> column 0 of cell row 1
        sta    ULA_DATA      ; bitmap byte for row 8 column 0

halt:    jmp    halt

```

Bare 6502 code stops simply by running an infinite loop, since the 6502 has no path to the system sentinel. The image executor at 0x1000 provides a small startup stub that sets up vectors and stack and jumps to start for .ie65 images.

26.9 What comes next

Chapter 27 covers the Z80, the other dominant 8-bit processor of the era. The Z80 has many more registers than the 6502 and a more powerful instruction set, but it shares the 64-kilobyte address space and similar Intuition Engine bank-and-MMIO mapping.